

Modern Logging Platform with Machine Learning for HPC

Marko Sladojevic

CERN, ALICE Event Processing Nodes

Supervisors: Lubos Krcal, Federico Ronchetti

September 2026

Abstract

The Event Processing Nodes (EPNs) form the ALICE computing farm that runs timeframe reconstruction. Logging on the EPNs is currently handled by *InfoLogger*, which carries only the messages a process sends to it, so log files written to disk and the system journal are never collected. The records that do reach *InfoLogger* are stored in a single central table, with no aggregation over time, one retention policy and no replication, and shifters inspect them by hand. The aim of this work is a single place to search and monitor every log of the farm without drawing resources from timeframe reconstruction.

For this purpose, a log platform named *loggy*, based on *Fluent Bit* and *OpenSearch*, was developed and deployed with *Ansible*. A collector on each worker reads every log source and stamps each line with its message template. Lines are then routed by severity: the bulk of the volume is indexed on the worker that wrote it and never crosses the network, while the lines that matter and every *InfoLogger* record go to a small replicated storage tier. Workers and storage nodes form a single *OpenSearch* cluster, so federated search is handled by default. Anomaly detection is applied both to the metrics and, through the message templates, to the log text itself. Alerts are merged into episodes and passed to *Alertmanager* for notification. Semantic search over the templates lets an operator find a message by its meaning.

Contents

1	Introduction	1
2	The problem and the constraints	1
3	The design	3
4	Why these choices	10
5	What the measurements say	16
6	The log text: detection and search	18
7	Anomaly detection on the metrics	23
8	Limits and what remains	27
9	Conclusion	29
A	Upstream roles weighed and not used	29

1 Introduction

Most logs an Event Processing Node writes never reach the one tool people use to read them. An Event Processing Node, an EPN, is one machine of the ALICE online farm. Its job is timeframe reconstruction: it assembles the data of one time slice from the experiment into events. The reconstruction software is *O2*, the ALICE Online-Offline computing system [3]. Log search must not compete with that job.

Logging today is *InfoLogger* [2]. A client library sends each process's messages to a local daemon, then to one central server. That road has three limits: one table for every message, no counts over time, and one unreplicated retention rule. The process logs on disk and the journal, the log of every system service, never take it (Figure 1).

We built *loggy*. It keeps the bulk of the logs on the worker, the EPN that wrote them, and still searches them as one. It runs on five staging machines, and one farm deployment of the cluster, the collectors and *OpenSearch Dashboards* passed every check.

Section 2 gives the constraints. Section 3 walks an error line, an info line, a dead collector and a quiet worker through it. Section 4 names the fact behind each choice, Section 5 what the measurements settle. Section 6 covers detection and search on the log text, and Section 7 anomaly detection on the metrics. Section 8 gives the limits and what remains.

2 The problem and the constraints

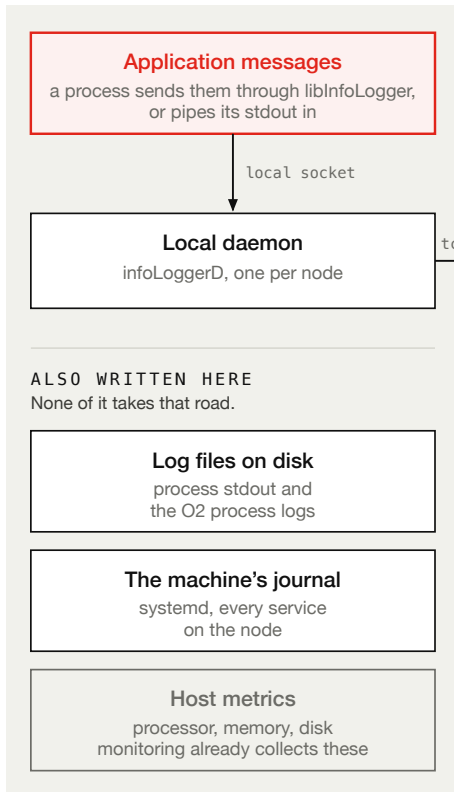
2.1 What a node writes

A worker holds five log families, and the *InfoLogger* records are a sixth source (Table 1). *DDS*, the Dynamic Deployment System, starts the reconstruction processes. The journal holds 2,500 to 10,500 entries a day.

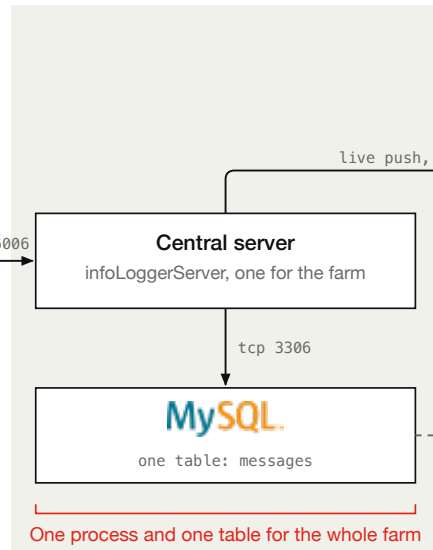
The survey of the real machines set two rules. First, the run directory sits on one 14 TB scratch disk, 93 % full, that every machine mounts over the network. Each collector tails only its own machine's directory. A tail over the whole disk lands every file on every machine. The mount raises no change events, so the tail polls. Second, one *O2* process prints raw terminal colour codes into its log, so every anchored pattern fails without a word.

The stored logs we replay are recorded farm logs, fed back through the collector at a chosen rate. They hold *InfoLogger*, *DDS* and the process tree, and no journal. The parser corpus is 45,596,613 of those lines from

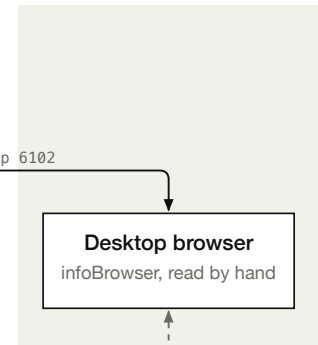
01 ONE EPN WORKER



02 CENTRAL, ONE FOR THE FARM



03 A DESKTOP



04 THE LIMITS TODAY

- **ONE SQL TABLE**
Every message in the farm lands in one messages table.
- **NO AGGREGATION**
infoBrowser lists single lines. It draws no counts over time.
- **ONE POLICY, ONE COPY**
One retention rule covers every message. Nothing is replicated.

Figure 1: InfoLogger carries only what a process sends it, and the process logs on disk and the journal never take that road.

Table 1: Only the *InfoLogger* records take the *InfoLogger* road, and the run orchestrator log reaches no collector.

Family	What writes it	How it reaches the collector	Tier it lands on
<i>InfoLogger</i>	Any process, through the client library and local daemon	A TCP socket	Storage, every record
<i>DDS</i>	The deployment system, one file a day in the run directory	Tail of the file	By severity
<i>O2</i> process tree	Every reconstruction process, one output and one error file each, in two line formats	Tail of the files	By severity
<i>InfoLogger</i> daemon log	The local daemon, one file that never rotates	Tail of the file	Storage, every line
Journal	<i>systemd</i> , every service and the kernel ring	The journal reader	By severity
Run orchestrator log	The orchestrator, on the shared infra machine only	Not collected	None

186 programs. *DDS* is 0.1 % of that corpus, and operations call it the largest family during data-taking. No run was active during either survey, so no live job log was seen.

2.2 What a worker carries

A real worker carries tens of *InfoLogger* records a second, not thousands. The *InfoLogger* archive, six months of dumps, holds 248,828,513 records from 312 hosts. In the busiest hour the median worker carried 23 records a second, the busiest worker-second 78, and the farm 9,781. These figures are floors: the client library cuts off a process at 1,000 messages a minute, so the loudest processes are hidden.

The soak, the project's measurement campaign of twenty-one rounds, tested one collector at 1,000, 20,000 and 50,000 records a second per worker. Those rates are a safety margin, not an archive figure.

2.3 The constraints

A worker gives the platform four of its 64 physical cores, eight of its 128 logical processors, and little memory. No bulk crosses the wire: each worker indexes its info-severity lines on its own disk. Physicists' staging runs occupy two of the three farm workers and a build pipeline wipes the third, so hard memory and processor caps apply.

2.4 What the platform must do

Table 2 lists the six objectives and the part of the design that meets each.

Table 2: One named element of the design meets each of the six objectives.

Objective	Design element that meets it
No bulk over the wire	A local index on every worker, written by a collector that talks only to the search node on its own machine
Split by severity	The collector routes each line by severity to the local index or the storage tier
Distributed	One <i>OpenSearch</i> cluster of two tiers, and every worker answers a search over its own index
Durable	Three machines, each holding a copy of every record above info and of every <i>InfoLogger</i> record
Workers stay cheap	Four physical cores, no cluster management on a worker, and health numbers pushed up rather than polled down
Fault tolerant	The cluster runs while two of the three storage machines agree, and a lost worker loses only its own local index

2.5 The 2025 reference design

loggy kept the shape of the 2025 reference design and changed its rules [1]. We kept its collector on every node and widened it from two log families to five. Its one *OpenSearch* cluster of two tiers, three index families, lifecycle policies and *Alertmanager* stayed. We changed its tier rule from source to severity, because a source rule ships one program's bulk and drops another's errors. We replaced *Grafana* with *OpenSearch Dashboards*, the store's own web interface, because detection and alerting come with the store [5]. We dropped its queue between the collectors and the store on the numbers in Section 5.3, and the aggregator it fed. A *Kafka* bus carries only the live lane, the second copy of every record that leaves a worker. We dropped *Kubernetes* for native system services.

3 The design

loggy is one *OpenSearch* cluster, split into two tiers by severity.

3.1 One *OpenSearch* cluster, two tiers

The worker tier runs on every EPN in the target design. The collector reads the six log sources. The stamper beside it names every line's template, the skeleton of the line with every variable token a wildcard, before indexing. The health sample pushes the collector's heartbeat up. The local *OpenSearch* data node holds one index per worker. The four share the worker's four-core budget of Section 2.3. Memory is a 384 MB cap on the collector and a 1 GB heap on the local node.

The storage tier is three replicated nodes, each a cluster manager and a data node. An index is split into shards, and a replica is a copy of a shard on another node. Two of the three nodes must agree, so the tier survives the loss of one.

The three storage nodes are also the control host, the background host and the shifter host. The control host holds the one web door every person opens. Behind it sit *Dashboards*, the maintainer's screen, and *Alertmanager*, which decides when a person is told. The control host also runs the poller, the roster of collectors that must run, the catalogue maintenance and the receiver. The background host runs the projector, which turns alerts into episodes, and the trend rollup. The shifter host serves the shifter view, the page the person on shift watches.

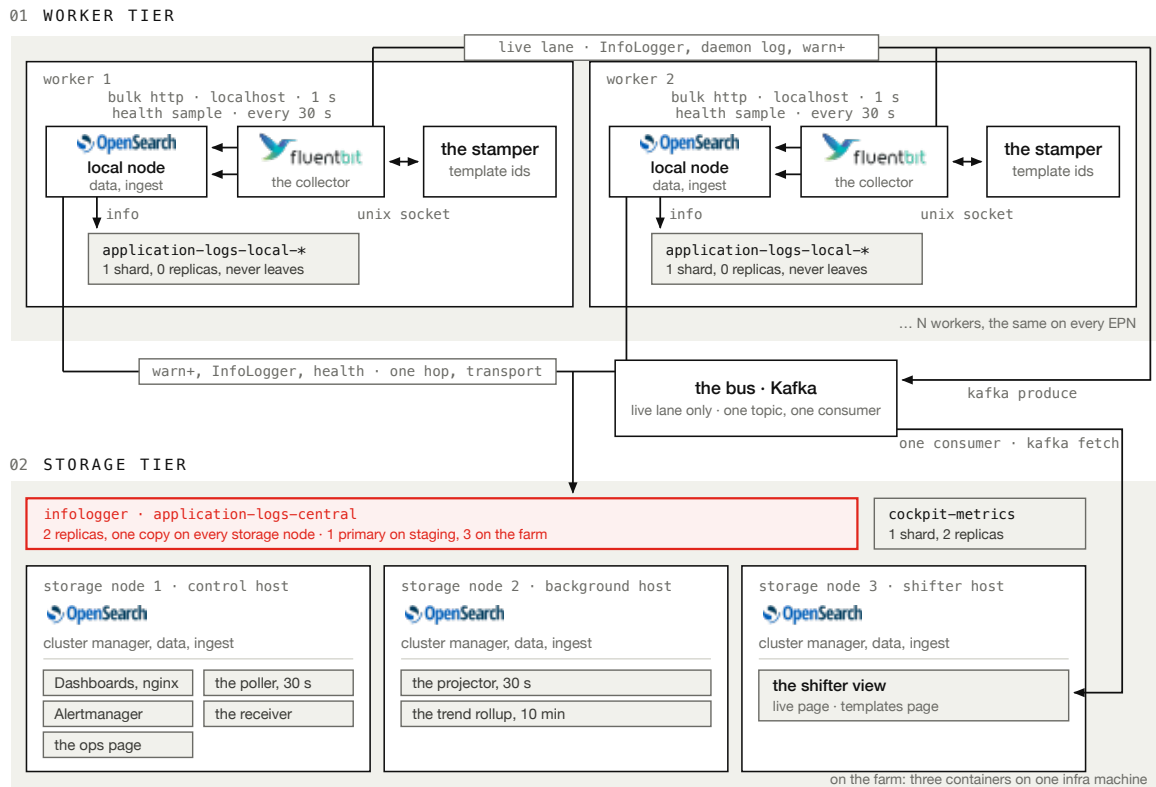


Figure 2: The bulk stays on the worker tier, *InfoLogger* and every line above info cross once to the storage tier, and every watcher pushes its numbers up.

Staging is five machines: two workers and three storage nodes. The farm pilot is three workers and three containers on one infra machine.

A *Kafka* bus carries the live lane only (Figure 2). The live lane is the second copy of every record that leaves the worker. That is every *InfoLogger* record at any severity, the daemon log and every line above info severity. Every collector writes it to one topic, and the shifter view is the one consumer. Moving the shifter view to another machine changes that one consumer and no collector configuration.

Table 3 lists the index families. The farm sets three primaries, one per storage node, because one primary funnels all *InfoLogger* through one machine. No farm run has carried real volume.

Table 3: The bulk sits in one unreplicated index per worker, and every other family sits replicated on the storage tier.

Index	What lands there	Tier	Shards and replicas	Retention
application-logs-local- <node>	info, debug and trace lines, one index per worker	worker	1 shard, 0 replicas	8 days
application-logs- central	warnings, errors, fatals, the daemon log, unparsed lines	storage	staging 1 primary, farm 3, 2 replicas	35 days
infologger	every <i>InfoLogger</i> record	storage	staging 1 primary, farm 3, 2 replicas	56 days
template-buckets-5m, -1h	counts per template version per worker and window	storage	1 shard, 1 replica	4 and 66 days
cockpit-metrics	health samples from collectors and the poller	storage	1 shard, 2 replicas	7 days by document

3.2 Walkthrough: one error line, from an EPN to a screen

An error line crosses the wire twice (Figure 3).

1. *The line appears in a file.* An O2 process writes an error line to its own log file, and the collector tails that file. The path gives the machine and the program, and the line gives the event time. The collector adds its own accept time and a document identifier unique to the record.

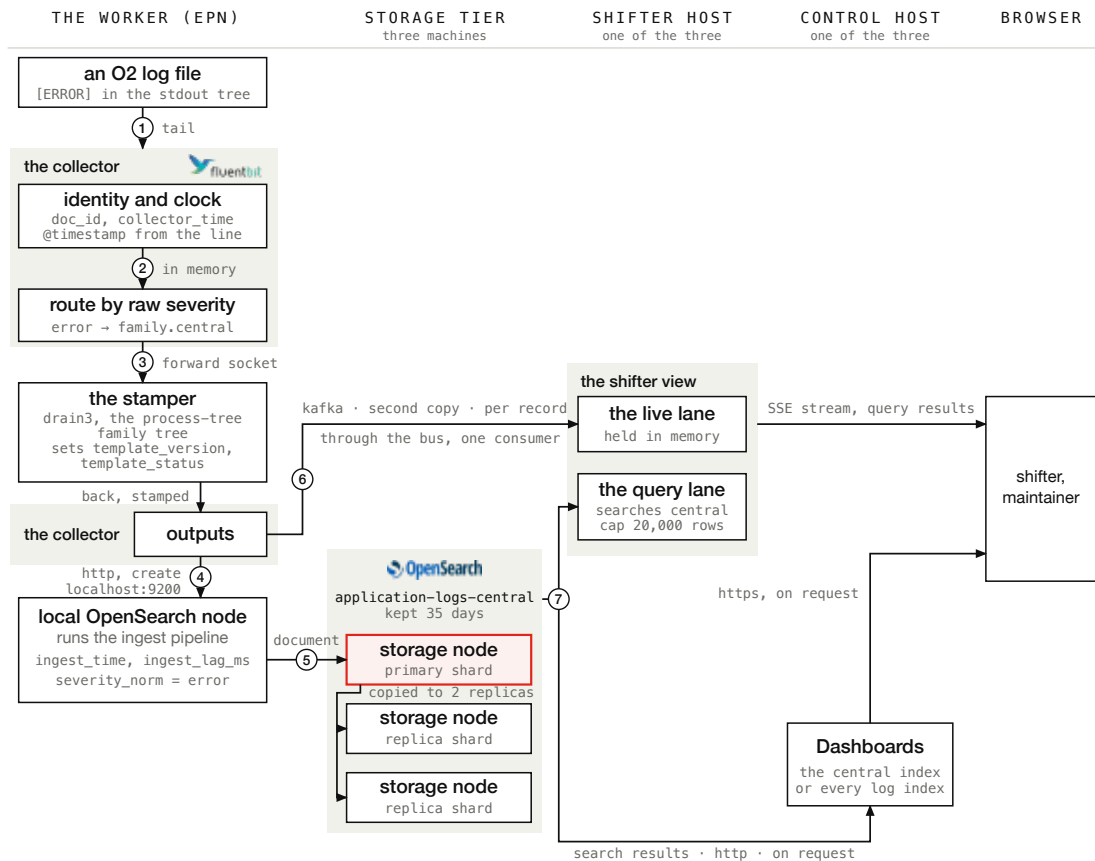


Figure 3: An error line is stamped on the worker, crosses the wire to the storage primary and two replicas, and again to the live lane.

2. *The raw severity decides the route.* A routing rule reads the severity as the line wrote it. Info, debug and trace take the local route, and this error takes the central one. A line the collector cannot parse also goes central, because a lost error costs more than an extra stored line.
3. *The stamper names the template.* The collector moves records in chunks. Each chunk crosses a local socket to the stamper, which masks the variable tokens and matches the line against its template tree. The chunk comes back with a template version and a status on every record. Only returned chunks can reach the store, so no record is indexed unstamped.
4. *Processed on the worker, stored on the storage tier.* The collector writes only to the store node on its own worker. There the ingest pipeline, a set of processors run on every document before storage, adds the ingest time and two lags. The first lag is the line's age when the collector read it, the second its trip into the store. The pipeline also maps every source's severity spelling to one vocabulary. The finished document then crosses the wire once, to the storage node holding the primary shard of application-logs-central. That node copies it to two replicas on the other storage nodes.
5. *A second copy goes to the live lane.* The collector also writes the record to the live-lane topic on the *Kafka* bus of Section 3.1. The shifter view consumes that topic, keeps the last 500 records in memory and streams them to every open browser. It never queries the cluster, so it keeps working when the cluster is down.
6. *Two screens can find it.* Discover, the search screen of *Dashboards*, finds the line in the central index. The shifter view's query lane searches the same index, capped at 20,000 rows. Both reads go to the storage tier and leave the worker alone.

3.3 Walkthrough: an info line stays on its worker

Most of what the farm writes is info level. This is the cheap path, and the storage tier never sees it.

1. *A DDS info line takes the local route.* DDS writes info lines by the thousand. The collector joins an event that spans several lines, such as a banner, into one record. The routing rule sends it to the local route.
2. *It is still stamped and processed.* Every family passes the stamper and the ingest pipeline on the worker, so the template counts cover the bulk too.
3. *It lands in a shard on the same machine.* Each worker has one index of its own, with one shard and no replica (Table 3). A placement setting pins that shard to the worker's own node, so the line never crosses the wire. That keeps the network free for reconstruction data. Under severity routing 96.94 % of the O2 process-tree family stays local.
4. *Three readers use it.* The trend rollup reads the index every ten minutes and six of the seventeen detectors every minute. The shifter view fetches a few real lines from it when a person inspects a template. If the worker dies, its raw lines die with it. The rollup rows, template counts and detector results survive, and the rules run on those.
5. *The live lane never sees it.* Across more than 200 workers the info bulk would saturate the wire and flood the live view. Only a search over the worker indices finds an info line, and that search is as slow as the slowest worker. Section 4.3 explains why we accept that cost.

3.4 Inside the collector

The collector stores nothing (Figure 4).

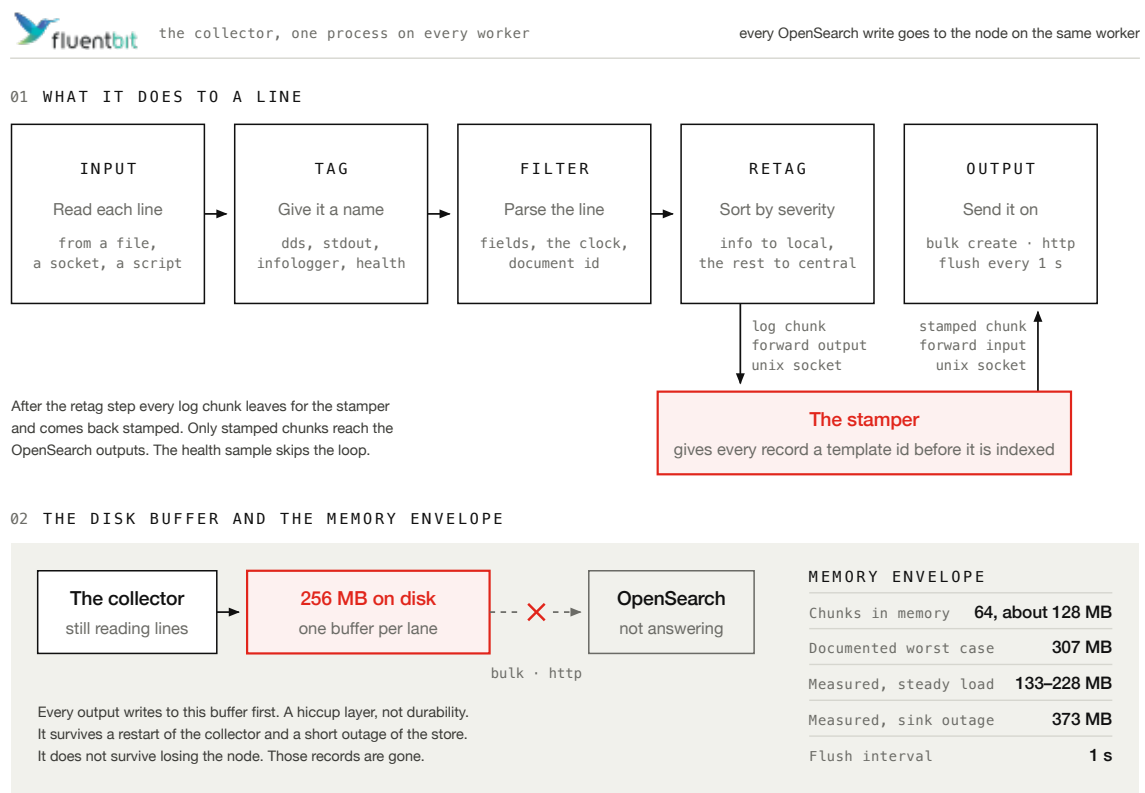


Figure 4: A line takes five steps inside the collector. The 64 chunks it holds in memory are about 128 MB of the 133 to 228 MB it uses under load.

Flushing every second instead of every 5 seconds cut the collector's processor cost by 24.5 % and its peak memory by 28.0 % (Section 5.2).

At most 64 chunks sit in memory, about 128 MB. The documented worst case is twice that with a 20 % margin, roughly 307 MB.

An output is the collector's connection to one destination. Each output has a 256 MB disk buffer: when the store does not answer, chunks spill to disk. At a real worker's rate it holds hours, but it is not durability: a lost machine loses its records.

A resend cannot duplicate. The document identifier, node id plus boot id plus a counter, is the record's id in the store. The output writes with create, which the store refuses when that id already exists.

3.5 Templates in band

Every record carries its template version before it is indexed (Figure 5). A template widens as the stamper sees more lines, and each state is one version.

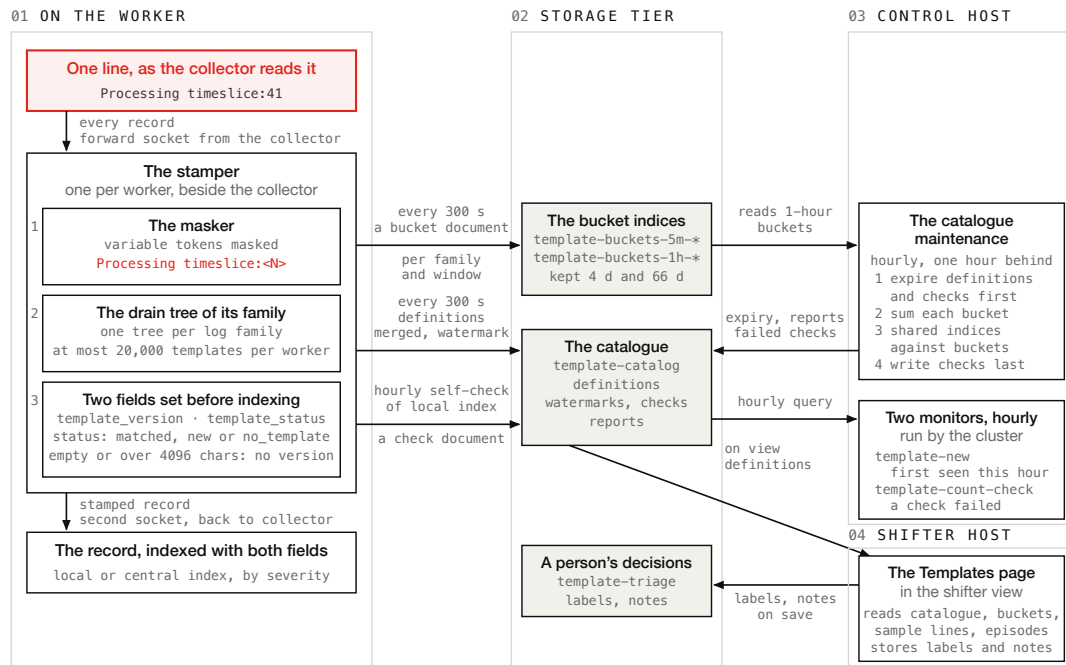


Figure 5: The stamper names each line on the worker, publishes counts every five minutes, and two hourly audits prove the counts add up.

The stamper runs *drain3*, a template miner [7]. A masker replaces varying tokens with placeholders, then one tree per family turns a literal that varies between lines into a wildcard. Every record leaves with a status: matched, new or no template. An empty message, or one over 4,096 characters, is indexed in full with no template.

The stamper holds at most 20,000 templates per worker, 4.7 times the 4,221 templates one tree mined from 55,963,050 archive lines (Section 5.5). The shipped service evicts the least recently used template at that ceiling. The soak showed that an evicted template rebuilt later resets its published count to one. The recommended change, to stop learning and keep counting, is open.

The catalogue is one index with one document per template version the farm has seen, plus the results of the checks below. Every 300 seconds the stamper publishes one bucket document per family and window: a total and a count per version.

Two audits run hourly. The stamper recounts the previous hour locally. The first check proves that no count is lost: up to 5,000 hourly buckets sum to their family totals. The second proves that no record is counted twice. Neither the central nor the *InfoLogger* index holds more records of a template version than the workers stamped.

Two hourly rules read the catalogue. One fires on a template catalogued in the last hour. The other fires on any failed check in the last two hours.

A search for a template must also find its older, narrower versions. Within one family and token count, a wider version covers a narrower one when, at every position, it holds a wildcard or the narrower's token. That relation groups the versions, so no separate identifier is stored. Templating is built for search and the catalogue. Two choices are open: catalogue maintenance as store policies plus monitors, and the bucket indices as a rollover alias.

3.6 Watching the platform

Everything that watches the platform is pushed up, never polled down. Every collector ships a health sample every 30 seconds. The poller samples the cluster at the same cadence and reads the roster of collectors that must run: the fleet.

Three lanes read the numbers: threshold monitors catch the cliffs we know, detectors catch unfamiliar shapes, and trend monitors catch slow drift. The numbers are health samples, and counts, error rates and lags from the log records.

A monitor is a saved rule the cluster runs on a schedule, writing an alert when its condition holds. An alert carries one of two tiers. A page wakes a person. A warn waits for one. A storage disk above 92 % raises a page, and above 85 up to 92 % raises a warn. An entity is one machine, or one log family on one machine: the thing a rule is about.

A detector is a scheduled Random Cut Forest model [8]. It learns a number's usual shape and grades each window from 0 to 1 by how unusual it is. Seventeen run, one model per entity. A volume detector reads a silent host as zero records, so it can grade silence. One monitor reads the grades and raises one page for the fleet when any passes 0.7. One forecaster, on disk fill only, trains on a week of hourly points and predicts a day ahead.

Every lane compares a recent window with the stretch of time before it, so what counts as normal moves with the farm. A threshold monitor checks every minute or every ten minutes against a fixed limit. A fast detector reads one-minute windows and a slow one 30-minute windows, and each point joins the last eight windows. A trend rule compares three ten-minute slices with the previous seven days. Tables 14 and 16 in Section 7 list every detector and trend rule. A detector keeps no fixed number of days. It holds a rolling sample of recent points and forgets older ones gradually, so a new level of traffic becomes normal after a while. The length of that memory follows from the detection plugin's defaults, which we neither changed nor measured. The trend rules cover what a detector forgets: they compare against a fixed seven days, so a slow drift that a detector absorbs still fires.

Thirty monitors run: 13 every minute and four every ten minutes on thresholds and detector results. Nine are trend comparisons, two run hourly on templates, two watch the projector and *Alertmanager* themselves. The two template rules are the detection on the log text today (Section 6).

The projector reads every alert and detector result every 30 seconds (Figure 6). A signal is one stored row: which rule or detector fired, on which entity, when. An incident is one condition on one entity, for example collector-down on one collector. An episode is one occurrence of an incident, with a start and an end. The projector stores each episode as one document. When the same incident returns later, a new episode opens. An episode closes after consecutive healthy windows: one for a threshold monitor, three for a trend monitor or detector. A grade above 0.5 opens an episode.

Alertmanager owns notification semantics only: grouping timers, silences and routing [9]. It forgets an alert five minutes after the last post, so the projector re-posts every open episode every 30 seconds. It holds a page 30 seconds and a warn five minutes, then sends one batch to the receiver. Nothing pages a human today.

The two monitors that watch the projector and *Alertmanager* post straight to the receiver, because that part can be dead. That is the break-glass path. The stale-projector monitor fires after ten minutes without a projector heartbeat. The other fires after the projector reports *Alertmanager* down for five minutes.

3.7 Walkthrough: a dead collector becomes a notification

A dead collector is silent, so the platform reasons from absence (Figure 6).

1. *Heartbeats stop.* While it runs, each collector pushes a health sample to cockpit-metrics every 30 seconds. The sample says whether the collector is up, and how many records it read, wrote, retried and dropped. When the collector dies, the samples stop, and nothing reports the death.
2. *The poller turns absence into a row.* Every 30 seconds the poller reads the roster, the list of collectors that must run. It writes one fleet row per rostered collector to cockpit-metrics. A collector with no

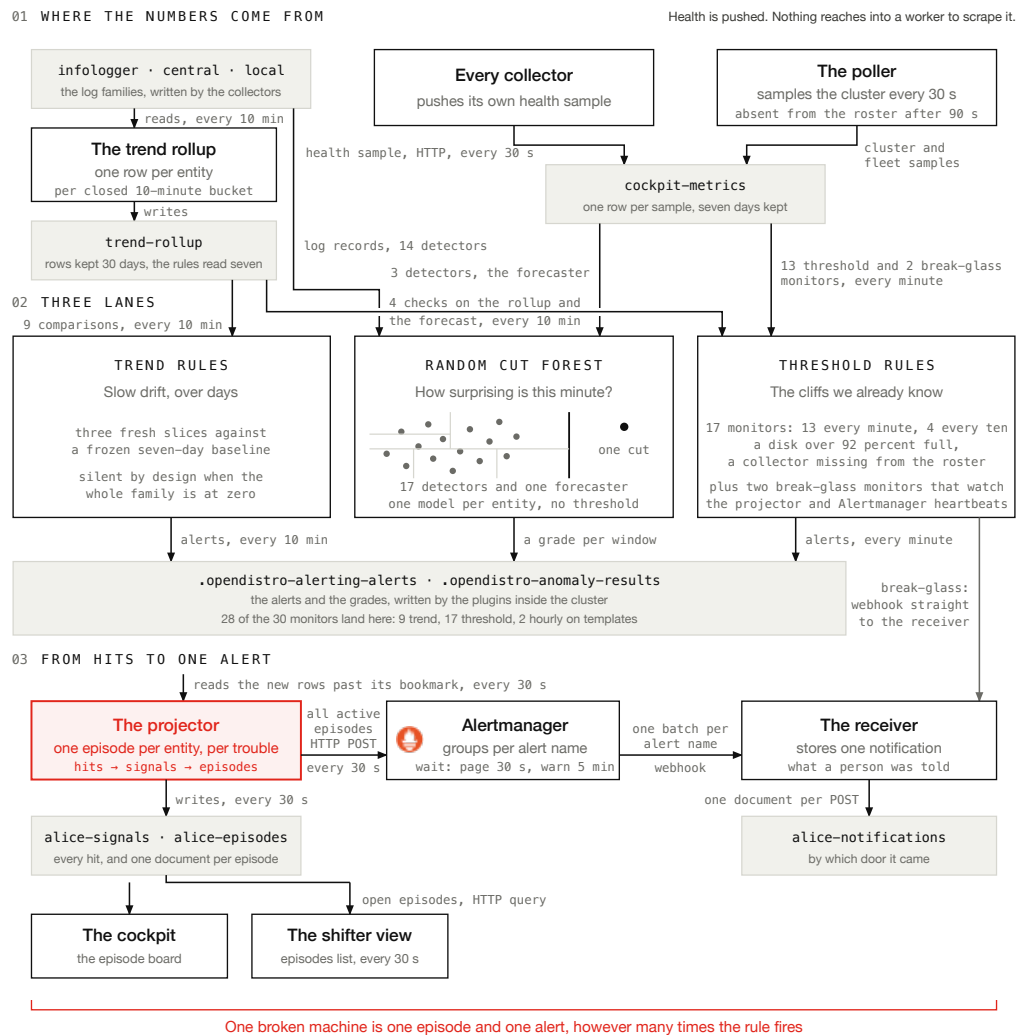


Figure 6: Pushed samples feed three lanes, the projector makes episodes, *Alertmanager* decides when a person is told, and the receiver stores what was sent.

sample in the last 90 seconds, three push periods, gets a row that marks its heartbeat missing. The health sample comes from the worker. The fleet row is the control host's verdict, written because the dead collector cannot write anything.

3. *The collector-down monitor fires.* Every minute the monitor reads the fleet rows of the last two minutes, one bucket per collector. One missing flag is enough for a page-tier alert that names the collector.
4. *The projector makes a signal and an episode.* Within 30 seconds the projector reads the alert. It writes one signal row to *alice-signals* with the collector, the alert name and the tier. It opens an episode in *alice-episodes*: collector-down on this collector, since now.
5. *The projector keeps Alertmanager told.* The projector posts the open episode to *Alertmanager* as one alert. *Alertmanager* keeps no database and forgets an alert five minutes after the last post. So the projector posts it again every 30 seconds, and posts an end time when the episode resolves.
6. *Alertmanager groups and delivers.* It treats the repeated posts as one alert and groups it with any other alert of the same name and collector. A page waits 30 seconds before the batch goes to the receiver.
7. *The receiver stores the notification.* The receiver writes one document to *alice-notifications* with the episode's identifier. Nothing pages a person yet, so that document records what a person would have been told.
8. *A person sees it.* The episode shows as a card on the maintainer cockpit and in the shifter view's list of open episodes.

The cadences bound death-to-document at about 90 to 240 seconds. The first missing flag arrives 60 to 120 seconds after the death. No run has timed it (Section 8.1).

3.8 Walkthrough: an EPN goes quiet, or loud

The trend rules never scan raw lines when they check. They compare small rows with seven days of the same rows (Figure 6).

1. *The rollup reads finished slices.* Every ten minutes the rollup service waits 120 seconds for late records. Then it reads the last three finished ten-minute slices from the log indices.
2. *It writes small rows.* The rollup writes one row per family, entity and slice to trend-rollup. An entity is a host, or a collector for the lags. A row holds the record, error and fatal counts, the 95th percentile of both lags, and the family's fleet total. The row identifier comes from family, entity and slice, so a recount overwrites the row.
3. *Silence becomes a zero.* A host that logged in the last 24 hours but not in this slice gets a row with a zero count. A commit row closes the slice once every row has landed.
4. *Nine rules compare with the last week.* Every ten minutes the nine trend monitors compare each entity's three latest slices with its own previous seven days. Without seven days of history, a monitor falls back to 24 hours.
5. *Volume rules compare shares.* A volume rule reads the host's share of its fleet, so a fleet-wide ramp cancels out. It fires when the share doubles or halves against the baseline in all three slices.
6. *Lag rules compare with the entity's own past.* The two shipping-lag rules fire when a lag passes twice its seven-day baseline. Trend alerts are warn tier.
7. *A dead family raises one page.* A separate monitor pages when a whole family writes nothing for 40 minutes, although it averaged at least 50 records a slice over the last day. The per-host volume rules skip that slice, so the dead stream raises one alert, for its family.
8. *A dead rollup raises its own page.* Another monitor pages when the rollup service has closed no slice for 40 minutes, because the trend rules then have no rows to read.
9. *The tail is the same as for any alert.* The projector turns each alert into a signal and an episode, as in Section 3.7. *Alertmanager* holds warn-tier alerts five minutes and batches them by rule name, and splits the shipping-lag batches by collector.

Which messages changed is the template lane's question (Section 3.5).

3.9 The surfaces

A person reaches all four surfaces through one web address and one account. *Dashboards*, the store's own web interface, holds the maintainer cockpit and Discover, its search screen for the archive [5]. The shifter view is separate. Its live page keeps what the collectors write to the bus: every *InfoLogger* record, the daemon log and every line above info severity. It never queries the cluster, so it holds when the cluster is down. Its Templates page shows the catalogue, counts and open episodes.

Two clocks run. Health samples and the log-counting detectors use the collector's accept time. Discover uses the time the line was written. The operator view close to *InfoLogger* is not built.

4 Why these choices

Each subsection below takes one choice in the design. It states the job the choice must do, the options we weighed, the fact that decided it, and the cost we accepted. Section 5 gives the measurements behind the numbers.

4.1 The collector: Fluent Bit

A collector is a small program that runs on every worker, beside the programs that write the logs. It reads each new line from a file, a socket or the journal. It parses the fields out of the text, decides where the line belongs and sends it on. It stores nothing itself. It shares a machine whose real job is reconstruction, so it must be small, and it must not need a second language runtime installed beside it.

Maturity and size decided it. Table 4 lists the four candidates. *Fluent Bit* [4] is written in C and needs no other packages unless a plugin asks for one. Its vendor gives about 450 KB of memory for an idle pipeline, against more than 60 MB for *Fluentd* [15]. Under our load it peaked between 133 and 228 MB, because the memory holds the chunks of records in flight, not the program (Section 3.4). *Fluent Bit* belongs to the *Fluentd* project, which graduated at the Cloud Native Computing Foundation (CNCF) in 2019 [16]. Graduated is the foundation's highest maturity level. It means a neutral foundation holds the roadmap, so no single company controls it.

Fluentd is the same project one size up, and the older of the two. Its core is *Ruby*, and it installs as a *Ruby* package that pulls in other packages. We did not want a *Ruby* package tree on every worker. *Vector* is small and fast, and published benchmarks disagree on which of *Vector* and *Fluent Bit* is smaller. Datadog bought the company that builds *Vector* in 2021, so one company decides its future [17]. *Telegraf* is a good agent for a different job. ALICE already runs it to gather machine and service metrics. It emits line protocol: a name, tags, numbers and a timestamp. That format carries metrics and has no place for the free text of a log line.

Table 4: Fluent Bit won on maturity and size. All four are open source, so the job and the governance decided.

Candidate	Language	Licence	Memory at rest, vendor's figure	Governed by	Verdict
<i>Fluent Bit</i>	C	Apache 2.0	about 450 KB	CNCF, inside the <i>Fluentd</i> project	chosen
<i>Fluentd</i>	C and <i>Ruby</i>	Apache 2.0	over 60 MB	CNCF, graduated 2019	a <i>Ruby</i> package tree on every worker
<i>Vector</i>	Rust	MPL 2.0	not published	Datadog	one company decides its future
<i>Telegraf</i>	Go	MIT	not published	InfluxData	emits metrics, not log lines

Why not a message queue at the edge. A message queue such as *Kafka* [11] moves records between programs. It does not make them. It cannot tail a file, parse a line or stamp a clock, so a forwarder still runs in front of it. The queue itself costs at least three more machines, its brokers, which must keep a quorum among themselves. Section 4.10 covers a queue behind the collector instead.

4.2 The store: OpenSearch

The store must do three things. It must index every field and the full message body, so a person can search for any word. It must count over time, for example errors per host per minute. It must also detect anomalies and raise alerts itself, so that we do not build and run that machinery beside it. An open licence was the floor, and detection decided it.

OpenSearch [5] is a search engine and document store. It began in 2021 as the open-source fork of *Elasticsearch*. It is licensed Apache 2.0, and since September 2024 the *OpenSearch* Software Foundation, part of the Linux Foundation, governs it [18]. Its standard package ships plugins for anomaly detection, alerting and notifications, so the detectors and monitors of Section 3.6 run inside the cluster that holds the logs. CERN IT's own monitoring service runs *OpenSearch*, so CERN already operates it.

Elasticsearch answers the same questions. One company holds it, and its source is offered under three licences: the Server Side Public License, the Elastic License and, since 2024, the AGPL [19]. Its machine-learning anomaly detection is in the paid Platinum and Enterprise tiers only [20]. With *Elasticsearch* the detection half of this project becomes a purchase.

ClickHouse is a column store under Apache 2.0, run by one company. It compresses logs well and counts fast, so it wins on disk. It ships no anomaly detection and no alerting, so it loses on machinery. We would

write, run and keep alive the 17 detectors and 30 monitors of Section 3.6 ourselves.

Loki, from *Grafana Labs*, is under the AGPL. It indexes a small set of labels per stream, such as the host and the application, and not the text of the line [21]. A search for a word, or a count of error lines per host, reads and filters every stored chunk in the time range. It ships no anomaly detection.

Table 5: OpenSearch is the only open candidate that indexes the message body and ships detection and alerting.

Candidate	Licence	What it indexes	Detection	Verdict
<i>OpenSearch</i>	Apache 2.0, Linux Foundation	every field and the message body	included, runs in the cluster	chosen
<i>Elasticsearch</i>	three licences, one company	every field and the message body	paid tier	detection is a purchase
<i>ClickHouse</i>	Apache 2.0, one company	columns, compressed on disk	none	we would build and run detection
<i>Loki</i>	AGPL, one company	labels only, not the text	none	a word search or a count is a scan

We accept two costs. Every index and every shard takes memory on the *Java* heap, so the number of indices is capped. Every plugin must match the exact release of *OpenSearch* it runs in, so an upgrade moves the whole set at once.

4.3 One *OpenSearch* cluster, not many clusters joined by cross-cluster search

Every worker keeps its own info lines on its own disk. A person still needs to search all of those disks at once. We make every machine part of one *OpenSearch* cluster, and ask it once.

Terms. An *OpenSearch* cluster is a set of *OpenSearch* processes, called nodes, that share one view of which indices exist and where their pieces live. That shared view is the cluster state. The cluster manager is the one node that keeps the cluster state and sends every change to the other nodes. In *loggy* only the three storage nodes can become cluster manager, and two of the three must agree to elect one. A worker's node holds data and answers searches. It never manages the cluster.

The first way: one cluster. All machines join one *OpenSearch* cluster. *OpenSearch Dashboards* sends a query to one node, which acts as the coordinating node for that query. In *loggy* that is a storage node. It reads from the cluster state which nodes hold shards of the indices asked for. It sends the query to each of them, waits for their partial answers, merges them and returns one result. The person writes one query over one index pattern, for example every application-log index, and never needs to know which machine holds what.

The second way: many clusters. Each worker runs its own small cluster, and a central cluster joins them with cross-cluster search [22]. Cross-cluster search lets a query on one cluster name indices on other clusters, written as cluster:index. Every remote cluster is registered by name on the central one, which keeps three open connections to each by default and forwards the query to all of them [23]. Table 6 sets the two ways side by side.

Why one cluster. First, a worker must not manage a cluster. With many clusters, each worker is a one-node cluster and so its own cluster manager, with its own cluster state, index templates, lifecycle policies and users. That breaks the objective that workers stay cheap (Section 2.4). Second, one cluster places shards and replicas itself across all machines, and the tier rule lives in index templates in one place. Third, one cluster has one set of users, roles and saved searches. Across clusters, a detector that reads a remote index needs a role that exists on both clusters and maps to the same user [24]. At 200 workers that is 200 copies to keep identical. Fourth, detection and alerting read every index with no remote step. Fifth, there is no federation layer of our own to write and keep alive. Nothing has to register a new worker as a remote cluster, or remove it when it leaves.

Scale does not rule cross-cluster search out, but it does not favour it. *OpenSearch* sets no hard limit on remote clusters. Each one adds connections and a round trip, and the project states that more remote

Table 6: One cluster keeps the workers cheap and the configuration in one place. Many clusters isolate faults and upgrades, at the price of a cluster manager on every worker.

	One cluster (chosen)	Many clusters, cross-cluster search
Who manages a cluster	three storage nodes	every worker manages its own
Tier rule, retention, templates	defined once, applied to every machine	copied to every cluster and kept in step
Users, roles, saved searches	one set	the same user and role on every cluster
Detectors and monitors	read every index directly	read remote indices through a role mapped on both sides
A machine joins or leaves	the cluster handles it	a remote cluster registered or removed by us
A bad query	can slow the whole cluster	slows only the clusters it reaches
Upgrades	one window for every machine	each cluster on its own schedule
A search over every worker	one fanout inside the cluster	one fanout across clusters, plus a connection per cluster

clusters mean fewer queries a second and higher latency [23]. We found no published deployment of cross-cluster search over hundreds of remote clusters. A single cluster of that size is documented: Amazon’s managed *OpenSearch* has run single clusters of up to 1,000 data nodes since late 2024 [25]. Those clusters use dedicated cluster managers, and ours do not.

What we accept: fanout. Every machine pays. A search over all worker indices reaches every worker, so every worker spends processor time on it. The answer is as slow as the slowest machine. Those machines reconstruct events, which is their real job, and log search must not compete with it. So we ask the cluster only when we must. The live lane covers watching what arrives now, and it never asks the cluster. A question such as “which machines saw this error in run 561234?” still has to ask all of them. Two more costs follow from one cluster: every machine upgrades in the same window, and one bad query can slow every machine. Three hundred data nodes in one cluster, one per worker in the archive, with managers that also hold data, is not tested (Section 8.2).

4.4 Two tiers, split by severity

An index is split into shards, and a replica is a copy of a shard on another node. A tier is a group of nodes that holds one kind of index. The worker tier holds each worker’s own info lines, one shard and no replica. The storage tier holds everything else, with a copy on each of its three machines.

We split by severity, not by source. Info lines are both the bulk and the lines least likely to matter, so they stay on the worker that wrote them and are disposable. Under this rule 96.94 % of the *O2* process-tree family stays local. Warnings, errors and every *InfoLogger* record are rare and are the lines people look for, so they cross the wire once and are copied three times. The journal follows the same idea through its own field. Each journal entry carries a priority from 0, an emergency, to 7, a debug message. Priorities 0 to 4, warnings and worse, go to the storage tier. Priorities 5 to 7, notices, information and debug messages, stay on the worker. The 2025 reference design split by source: one program’s logs local, another’s central [1]. That rule ships the whole info bulk of a chatty program across the network and keeps the errors of a quiet one on a single disk. It also needs four edits in step for every new log source.

4.5 Native system services on staging, containers on the farm storage machine

On staging every component runs as a system service. It is installed from the vendor’s RPM package, and *systemd*, the Linux service manager, starts it, restarts it and sends its output to the journal. A person operates it with the same commands as any other service on a CERN machine. The alternative was a container runtime on top of the *OpenStack* virtual machines, as in the local development stack. That is a second runtime to install, patch and debug. *Kubernetes* was planned in detail and dropped, because the EPN farm has no *Kubernetes*.

The farm gives the storage tier one shared infra machine, not three. The supervisor decided to keep the three-node tier as three *Podman* containers on that machine, rather than collapse it to one native node.

A container is a program run with its own file system and network ports, isolated from the rest of the machine. Cloning the tested three-node tier is cheaper than collapsing it and undoing that later. The same deployment code installs the RPM when a machine carries one node, and containers when it carries several. Tier settings, replica counts, templates and monitors do not change. Three replicas on one disk rehearse no fault tolerance. If that machine dies, the storage tier dies with it.

4.6 The surfaces: *OpenSearch Dashboards* and a standalone shifter view

OpenSearch Dashboards is the web interface of *OpenSearch*, built by the same project and released with it. It holds two of the four surfaces behind the one web address.

The maintainer cockpit is a dashboard in *OpenSearch Dashboards* for the people who maintain the platform. It answers one question: is the pipeline healthy? It shows which collectors are healthy, degraded or down, how many records each has shipped and lost, the cluster status, heap use and index sizes, and the open episodes and active alerts. A shifter never needs it.

Discover is the free-form search page of *OpenSearch Dashboards* [26]. A person picks a time range, types a query in the search bar and adds filters on fields, for example host or run number. Discover then lists the matching records and draws their count over time. One search box covers every log family at once. It serves a physicist with a real question about the archive.

We chose *OpenSearch Dashboards* because it ships the screens for anomaly detection, alerting and notifications in the same package as the store. There is no second product to install, and a person signs in once. The 2025 reference design used *Grafana* for display and alerting [1]. *Grafana* can read *OpenSearch* as a data source, but the detectors and monitors would still be configured through *OpenSearch*, so a maintainer would work in two interfaces. We did not compare *OpenSearch Dashboards* and *Grafana* on numbers.

The shifter view is the page the person on shift watches. It holds the live page, which shows what arrives now, and the Templates page, which shows the template catalogue, its counts and the open episodes. It is a standalone web page behind the same address, not an *OpenSearch Dashboards* plugin. A plugin must match the major, minor and patch release of *OpenSearch Dashboards* exactly [27]. Every security patch would force a rebuild of the plugin, and a failed rebuild would take the shifter's page down. The live page reads only the live-lane topic and never queries the cluster, so it keeps working when the cluster is down. A general query tool belongs in *OpenSearch Dashboards*, and Discover is that tool. The operator view close to *InfoLogger* is not built.

4.7 The stamper in band

A template is the skeleton of a log line, with every variable token replaced by a wildcard. The stamper is a small *Python* service beside the collector on every worker. It names the template of every record in band: on the record's path, before the record is indexed. It uses *drain3* [7], the same template miner that mined the archive.

We placed it on the worker so that no raw line leaves its machine to be named, and so that both tiers receive the template field once. The collector hands each chunk to the stamper over a local socket with *Fluent Bit*'s own forward protocol, and gets it back stamped. An HTTP hop was rejected because it replaces the record's own time with its arrival time.

We did not port the miner into the collector. *Fluent Bit* filters are written in C, *Lua* or *WebAssembly*, and none can host *drain3*, which is *Python*. A port would be a second implementation of the masker, the step that replaces variable tokens, and its exact output is the template's identity. Section 6.3 gives the reason every other template miner lost, and describes the masker rewrite. What the stamper costs beside a running collector is not measured.

4.8 Detection on numbers, inside the cluster

The platform must catch three kinds of trouble: known cliffs, unfamiliar shapes and slow drift. Threshold monitors, Random Cut Forest detectors [8] and trend rules each catch one kind (Section 3.6). A Random

Cut Forest learns the usual shape of a number from the stream itself, so it needs no labelled faults. ALICE has none. All three lanes run inside *OpenSearch*, as the detection and alerting plugins that ship with it, so they add no service to keep alive. No learned model reads the log text in the detection path. Section 6 gives the reasons, and Section 7 gives the detectors and rules.

4.9 Episodes, and *Alertmanager* as messenger

A firing monitor writes a new alert row every time it runs. A dead collector therefore produces thirty rows in thirty minutes, from several monitors at once. The projector reads all of those rows and groups the ones that describe the same trouble on the same thing into one episode, with a start and an end. Without it, a person would see the same alert every minute.

Alertmanager [9], from the *Prometheus* project, decides when a person is told and how often. It groups alerts into one message, holds a timer before it sends, lets a person silence a rule for a time, and routes each alert to a receiver. It stayed from the 2025 reference design. It keeps alerts in memory only. Its clients are expected to re-send every firing alert at regular intervals [28], and it forgets an alert five minutes after the last post. So the projector re-posts every open episode every 30 seconds.

Inhibition is an *Alertmanager* rule where one firing alert silences another that it causes, for example a dead cluster silencing every monitor that reads from it. It is built and ships switched off. All 22 cause-and-effect links between alerts are unproven, because no fault injection has shown that the cause always comes first.

4.10 Kafka between the collector and the store

Kafka [11] is a message bus: a set of brokers that store records in order and let many readers consume them at their own pace. The 2025 reference design put *Kafka* between every collector and the store [1]. A bus buys two things. Decoupling means a burst from the writers does not reach the store at once. Durability means records survive while the store is down.

The soak rejected it on both counts. For decoupling, one worker's stack sustains about 42,000 records a second. The busiest second any worker had in six months was 78, so the margin is 538 times. For durability, only *InfoLogger* records are lost in a storage outage, and the collector's 256 MB disk buffer holds 17.4 hours of them at a median worker's rate. The cost is real: three brokers on memory the staging machines do not have, and the tier rule would then live in two places. Section 5.3 gives the four conditions that reopen the decision.

4.11 Kafka for the live lane

The live lane is the second copy of every record that leaves a worker. Each collector writes it to one *Kafka* topic, and the shifter view consumes it. The bus carries the live lane only, and it exists to decouple the collectors from the shifter view. The collectors' disk buffers already survive a restart, so durability alone would not justify it. Because the shifter view is the topic's one consumer, moving it to another machine changes one place instead of the configuration of more than 200 collectors.

It must be Apache *Kafka*, because CERN requires fully open-source software. *Redpanda*, the usual alternative, is under the Business Source License, which is source-available and not open source [29]. The bus is the collector's primary live-lane output. It cannot be a fallback, because the collector held at release 4.0.1 has no route for a failed output.

4.12 Deployment: Ansible

The agent decided it. An agent is a program the deployment tool installs and leaves running on every machine. *Ansible* [10] has none. It connects over SSH when a person starts a run, pushes the configuration and leaves. CERN hosts take no long-running agent from this project, so nothing new runs on them. The same tool also creates the virtual machines. Table 7 lists the candidates.

Puppet with *Foreman* is the CERN standard, and ALICE build machines already run it. It repairs drift by itself on a timer. Each machine needs a *Puppet* agent and a certificate signed by the *Puppet* server [30]. That

Table 7: Ansible was the only candidate with no agent that also creates the machines.

Tool	Agent on every machine	When it runs	Creates machines	Verdict
<i>Ansible</i>	none, over SSH	when a person runs it	yes	chosen
<i>Puppet</i> with <i>Foreman</i>	agent and signed certificate	on a timer	yes, with <i>Foreman</i>	right for the farm, wrong for five machines
<i>Salt</i> , <i>Chef</i>	agent and a server	on demand or a timer	no	an agent on every node
<i>Terraform</i>	none	when a person runs it	yes, and nothing else	a second tool to configure them

is likely right for the farm. It is wrong for five machines that one person deploys by hand. *Salt* fans out fastest, but only at farm scale. *Chef* needs a second language. Both put an agent on every node. *Terraform* builds machines well, and its vendor states that it is not a configuration management tool [31]. We would need a second tool to configure the machines. We did not consider shell scripts, because a script does not check the state a machine is left in.

Three design rules followed. Placement is data: one inventory file says where each service runs, and moving a service to another machine changed no code. Every step converges: a failed run is simply run again, and monitors and detectors are created by name, never twice. Twelve checks fail the deploy rather than report success.

4.13 Cockpit metrics beside *Telegraf* and *Mimir*

The cockpit needs numbers about the log pipeline itself: is each collector alive, how far behind is it, how full is the cluster. The poller is a small service on the control host that samples the cluster every 30 seconds. Each collector pushes its own health sample, so nothing reaches into a worker to ask. The roster is the list of collectors that must run, and the poller marks any rostered collector that falls silent. A dead collector sends nothing, so only the roster lets the platform notice it.

CERN already runs *Telegraf* and *Mimir* for machine metrics. *Mimir*, from *Grafana* Labs, stores *Prometheus* metrics for the long term. Our cluster and node samples duplicate part of that estate. The roster and the absence logic have no equivalent there, so they stay. Whether the existing estate takes over the overlapping samples is open.

4.14 Shared code as one package

About 31,000 lines of *Python* run the services. The code that more than one service uses lives in one versioned *Python* package that every service installs, and no service keeps its own copy of a shared module. A file is in the package when more than one service uses it, another module imports it, it needs third-party libraries, or it needs tests. We rejected a port of the services into *OpenSearch* plugins, because a plugin must match the exact *OpenSearch* release and needs a *Java* toolchain. Appendix A lists the existing deployment roles we weighed and did not use.

5 What the measurements say

The numbers settle what the stack costs a worker, its ceiling, and what templating costs. Every figure comes from a laptop rig. Shapes, knees and rankings transfer to the farm. Absolute rates and core-seconds do not.

5.1 The instrument and its limits

The rig runs the whole stack in containers. The instrument sums every container's processor time into core-seconds, one core busy for one second. Cost is core-seconds per million records.

Two rigs exist, never averaged. The healthy rig tested the collector's settings at 20,000 records a second.

The re-run rig, the same laptop slowed, offered 5,000. An arm is one setting under test, and a cell is one arm at one rate.

The noise floor is the spread of three control runs: 1.61 % at 20,000 records a second and 9.48 % at 1,000. Cost per record falls as rate rises: the collector alone costs 75.32 core-seconds per million at 1,000 a second and 11.19 at 20,000.

The rig saturated unnoticed after that first pass, so we retracted every later measurement. A 47-cell re-run left one product change: the flush interval.

5.2 The collector under load

Against a sink that always accepts, the collector's memory sat between 133 and 228 MB on two cores, up to 53,000 records a second. In a sink outage with the shipped 256 MB buffer it peaked at 373 MB, under its 384 MB memory line.

The flush interval is how often the collector pushes buffered records out. Table 8 comes from three alternating runs. Across eight flush values from 0.125 to 10 seconds, whole-stack cost is a U with its floor at 1 second, best to worst 17.9 % apart.

Table 8: Flush 1 second beats the shipped 5 seconds on every measure, at 5,000 records a second on the re-run rig.

Measure	Flush 5 s (shipped)	Flush 1 s (chosen)	Change
Whole stack, core-seconds per million records	108.40	104.67	−3.4 %
Collector alone, core-seconds per million records	36.70	27.70	−24.5 %
Collector peak memory, MB	87.1	62.7	−28.0 %
Live-lane latency floor, s	5	1	

Every threading arm cost more against the 1.61 % floor: a second collector process 7.4 %, a thread per source 11.5 %, and parsing moved into those threads 26.5 %. One single-threaded loop does 89.3 % of the collector's processor work, so threads elsewhere cannot help it.

The live lane of Section 3.1 is the largest cost any setting adds: 41 to 68 % more collector processor time on the healthy rig.

A fifteen-minute sink outage at 20,000 records a second loses 66.7 % of *InfoLogger* at the buffer cap. That buffer holds 865,674 *InfoLogger* records of 310 bytes, and *InfoLogger* is 60 % of a worker's mix. At a median worker's 23 records a second *InfoLogger* is 13.8, and the buffer covers 17.4 hours. At the busiest worker-second, 78, it is 46.8 and the buffer covers 5.1 hours.

The stack sustains about 42,000 records a second. It holds 50,000 for two minutes with zero loss. The storage tier runs at 86 % of its four cores. The decline starts near six million documents, where index merging cuts throughput, so the ceiling is the storage tier's.

Overload is safe. At 82 % of the buffer cap the collector absorbs the shortfall and loses nothing. At the cap it discards oldest-first, 1.8 % of records, without error, then drains in 263 seconds.

Two settings make no difference. On the re-run rig a 2 GB heap moved cost 2.1 % against a 6.7 % floor, so 1 GB ships. Core placement between collector and store moved it 2.7 % against a 5.5 % floor. The rig always pinned four exclusive cores, so sharing them with reconstruction is not measured.

5.3 Kafka between the collector and the store, decided on numbers

The soak rejected a queue between the collector and the store on three counts. Decoupling: the stack sustains 538 times the busiest worker-second and 4.3 times the farm's peak second of 9,781. If the archive under-counts by ten times, the margin is 54 times. Durability: the collector's buffer holds hours at a real worker's rate. Cost: three brokers and a second system to run, on memory the machines do not have.

Two rates reopen the question: a sustained worker rate above about 1,000 records a second, or one within 50 times of 42,000. Two needs do too: a second consumer, or an outage longer than the buffer holds. The live-lane bus is separate.

5.4 The other sources and the real node

The collector now reads three log sources it had never read before. They are the second process-tree format, written by the data-distribution processes, the *InfoLogger* daemon log and the system journal. Severity now comes out of 99.83 % of process-tree lines, against a 99 % gate, and 96.94 % of that family stays on the worker. The first fixture run sent 41.8 % of the corpus to the storage tier with no severity. A parser anchor missed a date the replay prepends.

Four facts in the design now come from a read-only census of four farm machines. Four installations run there, on three releases where we assumed two. The storage machine's release is two majors behind and untested. A worker's journal is small beside millions of process-tree lines, so the collector reads all of it.

The collector passes the fixture and restart checks on the three farm releases and on one newer. Release 5.0.8 is blocked: it loses every byte appended after rotation, in five of five runs. The rotation check now rotates only after a line written before it has reached the output, so the loss shows on every run.

No regex change in the collector is worth making on cost. Two identical configurations differ by up to 19.3 % in processor time, so the rig cannot resolve a change below about a fifth. For that reason an external review retracted every collector regex cost figure.

The product now writes with create under a stamped document identifier, so a retried bulk write no longer duplicates every record. That was one of 99 defects fixed in the test rounds: 32 in the instrument, the rest in the collector, the stamper and the catalogue.

5.5 Templating

Templating is built, and the September rewrite of the steps around the miner made it 31 to 57 % cheaper per family (Table 9). Section 6.3 gives why *drain3* was chosen, the recipe per family and the masker rewrite.

The template count grows with runs sampled, not lines. One run of each family on 45,596,613 lines gave 3,011, a floor. Sixteen runs and eighty *InfoLogger* partitions gave 4,092. One tree carried across that corpus and two further archive pulls, 55,963,050 lines, gave 4,221. New templates arrive at 48.9 per million, about one a second at 20,000 records a second.

Splicing the stamped fields into the record's own bytes cut transport 15 %. Transport is roughly a quarter of the collector-to-stamper hop, which fell 6.5 % on *InfoLogger* and 4.0 % on the process tree. Repeated key names are 32 % of that wire, free on a Unix socket, so we rejected protobuf and gzip unmeasured.

Table 9: Every row is a ratio or a ceiling measured on one host, and the rewrite left the output byte-identical.

Figure	Value	Reference
Pipeline after the rewrite	31 to 57 % cheaper	byte-identical on 8,961,245 lines
Templates, one tree across 55,963,050 lines	4,221	3,011 from one run per family
New templates, steady state	48.9 per million	about one a second at 20,000 records a second
Collector-to-stamper transport after splicing	15 % cheaper	byte-identical
Growth ceiling	20,000	4.7 times the 4,221 mined, 208.1 MB against 512 MB

5.6 Injection tests for the detection chain

Seven scenarios score whether a signal, an incident and a notification followed one thing broken on purpose. A fault agent on each worker kills its collector, and one beside the projector stops only the projector. A poison replay floods the indices with marked fake errors. The stopped-projector run passes only if one stale-projector notification takes the break-glass path of Section 3.6. The scorer checks presence and order, not time, so the bound in Section 3.7 follows from cadences and is not measured.

6 The log text: detection and search

The platform detects on the log text through templates. The stamper turns every line into a message form, and the detectors watch those forms: a form never seen before, and how often each known form appears.

What we rejected is detection on raw lines and on embeddings. Figure 7 shows where each step of the standard route ended.

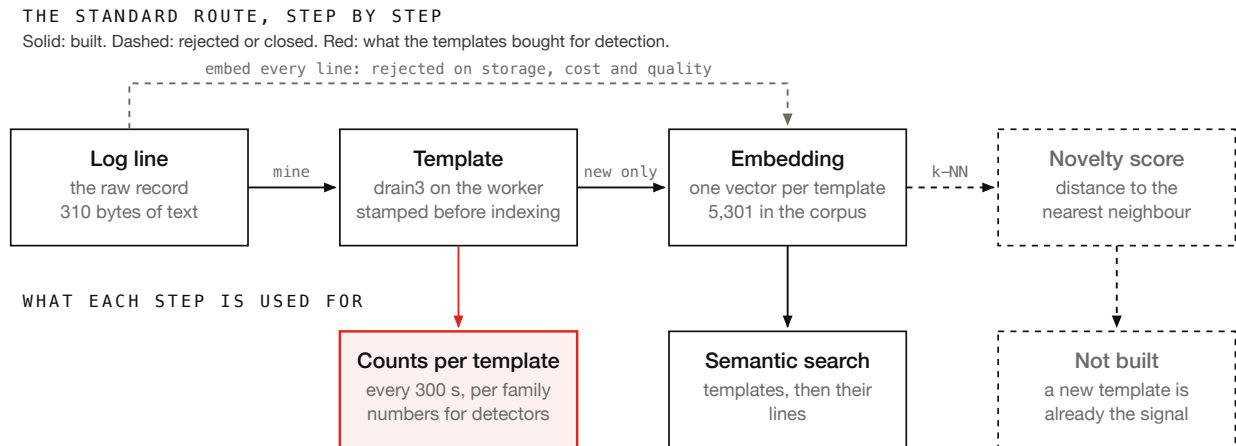


Figure 7: Templates are built and feed counts to the detectors, embeddings serve search only, and the novelty score on vectors is not built.

6.1 The standard route

Anomaly detection on the log text asks one question: is this message unlike every message seen before? The usual answer has three steps.

1. **Templating.** Replace the varying parts of a line with placeholders, so millions of lines collapse into a few thousand forms. Processing timeslice 41 becomes Processing timeslice <NUM>.
2. **Embedding.** Compute a vector for each form that stands for its meaning. Two messages that say the same thing in different words get vectors close together.
3. **Nearest-neighbour search (k-NN).** For a new message, find the closest vectors already seen. If even the nearest one is far away, the message is *semantically new*: it says something no earlier message said.

Each step can run on the worker or on the storage tier, per line or per template. Those two choices decide the cost.

6.2 Embed templates, not lines

Decision: one vector per template, computed once, and none per line. Table 10 sets the two side by side.

Table 10: A vector per line costs ten times the logs it describes, and the only model fast enough to run per line does not beat plain word search.

	Vector per line	Vector per template
Size of one item	3,072 bytes, ten times the 310-byte record	about 40 kB with its reranking share
Total	about 764 GB for six months of archive, against 77 GB of text	about 211 MB for 5,301 templates
Static model at 23 records a second (median worker)	0.13 % of one core	negligible
Static model at 20,000 records a second (design rate)	1.1 of the worker's four cores	negligible, about one new template a second
The model that wins on quality	113 templates a second on a laptop, so out of reach	47 s for the whole corpus

Storage. Per item a template is dearer, about 129 times a line. Thousands of lines share one template, so the total is what counts.

Worker cost. Encoding every line with a static model costs nothing at a real worker's rate. At the design rate it takes over a quarter of the four logging cores for one feature. Keeping that work off the worker

matters at the rate the stack is built for, not at today's rate.

Quality decides it. Only a static model is cheap enough to run per line, and on the sealed set it scores 0.353 against 0.371 for plain word search (Section 6.5). The model that does beat word search is far too slow per line.

What we keep. An operator reaches a line in two steps: semantic search over templates, then the store's ordinary search over the lines of the template found. The second step is free, because every record already carries its template version.

6.3 Choosing the template miner

Decision: *drain3* [6, 7], with one tree and one recipe per log family, behind a rewritten masker. No parser can be scored for accuracy on our logs. The correct template for an *O2* line is the format string in the *O2* source, and nobody has extracted those. Cost, memory, streaming and readability decided instead. Table 11 gives the reason each candidate lost.

Table 11: Every other parser lost on cost, streaming or readability, and the store's own command works at query time.

Candidate	Why it lost, measured on the same archive lines
drain3	chosen: streams, 26.3 MB, readable once configured per family
<i>PIPLUP</i>	2.56 times the cost and 7.8 times the memory of <i>drain3</i> ; one placeholder type for every variable
<i>LogLSHD</i>	slower than the <i>Drain</i> it claims to beat; needs a threshold tuned against ground truth we lack
<i>KELP</i>	crashed three times on real lines, ships no masking, lost on every column once patched
<i>IPLoM, AEL</i>	batch algorithms: they cannot stream
<i>LEA</i>	cheap, but 78.3 % of its templates keep one literal word or none
<i>SHISO, LenMa, Logram</i>	4.8 to 13.3 times the reference <i>Drain</i> 's cost
<i>Spell, LogMine</i>	no result in ten minutes
<i>EFParser</i>	calls a language model for every line
The store's patterns command	mines at query time over stored records (reasoning, not measured; see below)

Why one recipe per family. A recipe sets which separators get padded with spaces and how similar two lines must be to share a template. The three families want different answers to the same setting, as Table 12 shows. One recipe per family costs what one global setting costs, 17.57 against 17.65 weighted core-seconds per million lines. It cuts *contentless templates*, where every token is a placeholder, from 44.9 to 10.4 % on *InfoLogger*.

Table 12: Each family wants a different separator setting, which is why each gets its own recipe.

Family	Padded separators	Why
<i>InfoLogger</i>	= ; :	full of key: value, so it wants the colon
Process output	= ;	the colon splits real words
<i>DDS</i>	=	a shell command line uses ; between commands

Why the masker was rewritten. The masker replaces numbers, addresses and identifiers with placeholders before the tree sees a line. Without it, mining is 46 times dearer, because every distinct number becomes a distinct token and the tree grows 92 times the clusters it should have. It was also 74 to 87 % of the mining cost. We rewrote seven regular expressions to reach the same output by a shorter path, which made masking 85 to 87 % cheaper with zero differences on 3,000,000 lines per family. The September rewrite of the steps around the miner then made the pipeline 31 to 57 % cheaper per family, byte-identical on 8,961,245 lines, with the cluster of every line compared in order.

Why not the store's own templating. The store's patterns command mines forms from records already stored, at the moment someone queries. It cannot stamp a template on a record before indexing, it cannot run on the worker where the bulk stays, and it rebuilds its forms on every query instead of keeping one catalogue for the farm. This judgement comes from how the feature works. We did not measure it.

6.4 Templates are the text detector

Templates turn each line into a form, and a form can be counted. That is how the platform detects on the text. Two signals come from them, one built and one half built.

A template never seen before (built). The stamper marks every record as matched, new or without a template, and an hourly rule fires on any template catalogued in the last hour. New templates arrive at 48.9 per million lines in the steady state, about one a second at the design rate.

Counts per template (published, not yet read). Every worker publishes a count for each template version every 300 seconds. That gives a time series per message form, which the three lanes of Section 3.6 can read the same way they read a whole family:

- a message that stops, when a component has gone quiet;
- a message that floods, when one form suddenly dominates its family;
- a message whose rate drifts from its own week-old baseline.

Read that way, they would enter the same moving windows as a family's counts (Section 3.6): a per-template detector would compare each minute with a rolling sample of recent minutes, and a trend rule would compare the last half hour with the previous seven days. No detector or trend rule reads these counts yet. Wiring them in is the next detection step (Section 8.3).

6.5 Embeddings are for search

Decision: the template vectors serve semantic search only, and not nearest-neighbour novelty detection. Section 6.6 gives the reasons. *DenseOn* scans all 5,301 template vectors exactly, and *LateOn* reranks the top 30. The search ships enabled. The service is capped at 384 MB. The model's memory peak was seen in one local run and is not recorded.

How the benchmark was built. We scored every system on nDCG@10, which rewards relevant templates near the top of the first ten results and reaches 1 when the order is perfect [12].

- **Corpus:** 5,301 template groups mined from 56,628,579 archive lines.
- **Development set:** 97 intent groups and 130 questions, written by two machine writers from real templates.
- **Judging:** four system families pooled their top twenty for 6,410 candidates. Six machine assessors graded every candidate. The lowest weighted agreement between two assessors was 0.761, against a threshold of 0.60 fixed in advance.
- **What counts as real:** 0.0320 nDCG@10, the measured noise between two word-search baselines. A smaller gain is not acted on.
- **Sealed held-out set:** 62 intent groups written before any model was compared, then opened once.

Figure 8 gives the results. The ladder runs cheapest first, from BM25, the store's word-matching score, to *LateOn*, which scores a question against a template word by word.

Why the gap is so large. In 47.1 % of relevant question-and-template pairs, the two share no word after analysis. Operators ask in their own words, and developers wrote the templates in theirs. A dense model closes that gap and word search cannot.

What we embed. Writing a template's source and program in front of it gives the largest single gain in the benchmark. We tested three texts per template: the template alone, the template after its source and program, for example source=dpl program=gpu-reconstruction, and that text with severity and detector added. Source and program gained 0.168 for *DenseOn* and 0.092 for the static model. Severity and detector added nothing, because 35.4 % of templates carry no severity. For word search, no text beat another by the 0.032 bar. The shipped search embeds the source, the program and the template.

Why *DenseOn*, then *LateOn*. *LateOn* scores best, but searching the whole corpus with it needs a store of about forty vectors per template, and no such store was built or verified. Reranking only *DenseOn*'s top 30 takes 0.6 ms and keeps most of the gain: 0.685 against 0.634, with a bootstrap interval on the gain of +0.028 to +0.080. Bigger models bought nothing. The 0.6-billion-parameter model matched *DenseOn* and took 33 times as long to encode the corpus. Table 13 lists what else we tried.

DEVELOPMENT SET

97 intent groups, 130 questions, every candidate graded. The ladder, cheapest first.

System	What it is	nDCG@10	encode corpus
BM25	the store's word-matching score	0.396	no model
BM25F	BM25 over weighted fields	0.471	no model
potion-base-32M	static model, one vector per word	0.471	–
potion-retrieval-32M	static model, tuned for search	0.479	1.5 s
all-MiniLM-L6-v2	small transformer	0.494	9 s
Qwen3-Embedding-0.6B	large dense model	0.629	1,556 s
mDenseOn	multilingual dense model	0.630	569 s
DenseOn	dense retrieval model	0.640	47 s
LateOn	late interaction, per-word scores	0.695	–

SEALED HELD-OUT SET

62 intent groups written before any model was compared, opened once. Natural-language questions.

System	What it is	nDCG@10
DenseOn	dense retrieval model	0.689
BM25F	BM25 over weighted fields	0.371
potion-retrieval-32M	static model, tuned for search	0.353

THE SHIPPED CONFIGURATION

One deterministic run: exact cosine scan over all 5,301 vectors, no approximate index.

System	What it is	nDCG@10
DenseOn alone	scan, top 10	0.634
DenseOn + LateOn	scan for 30, rerank 30 in 0.6 ms	0.685

Figure 8: The dense model beats word search on the sealed set by 0.318, and reranking 30 candidates adds a further 0.051 on development questions.

Table 13: Every idea tried on top of the dense model either lost or did not clear the bar the project set for itself.

Idea	Effect, nDCG@10	Verdict
Source and program written before the template	+0.168	kept: the largest single gain
Severity and detector added as well	no gain	dropped: 35.4 % of templates carry no severity
Word search fused with the dense model, unweighted	−0.048	dropped: worse than the dense model alone
The same fusion with tuned weights, or three-way with <i>LateOn</i>	+0.008	dropped: below the 0.032 bar
Word and dense candidates pooled, then reranked by <i>LateOn</i>	+0.059	no gain over reranking dense candidates alone
Approximate vector index	−0.020	dropped: results changed after each rebuild
Multilingual model	no gain	dropped: the logs are English
Static model distilled with 3,202 ALICE words	+0.039	not promoted: fails the correction for 15 comparisons
The same model further trained on 15,647 ALICE lines	+0.0002	dropped: the vocabulary carries the gain
Static model trained on our own relevance grades	+0.063	a lead only: measured on unseen questions, one split of 43

Hybrid search. Adding word search to the dense pipeline never gained as much as the 0.032 bar. We tried six hybrids: rank fusion, unweighted and with tuned weights, two score fusions, a three-way fusion that adds *LateOn*, and a candidate pool that *LateOn* reranks. Unweighted fusion loses, because 28.6 % of the relevant templates in the dense top ten are absent from word search’s top 200. In the final judging round, which scored every arm on the same pool, tuned fusion and the three-way fusion beat the dense model alone by 0.008. The reranked pool scored 0.694, and reranking dense candidates alone scored 0.695.

A static model built for our logs. No static model reached the dense model, including tables we built for our own logs. A static model such as *potion* is a lookup table with one vector per token. It averages the vectors of a line’s tokens and runs no transformer, so it is the only model cheap enough to run per line (Section 6.2). We expected a table built for ALICE logs to match or beat a general dense model, because the log vocabulary is narrow. We built our own table instead of reusing *potion*’s. We distilled it from a transformer over 3,202 ALICE words, trained it further on 15,647 ALICE lines, and finally trained it on our relevance grades. Figure 8 shows the stock *potion* table at 0.479 on development questions, against 0.640 for the dense model. Our distilled table gains 0.039 over stock, and all of that gain comes from the ALICE vocabulary. The gain fails the correction for 15 comparisons. Training on log text adds 0.0002. Training on relevance grades scores 0.533 on 43 questions it never saw, against 0.470 for our distilled table. That closes about half the distance to the dense model, but on one small split, so it stays a lead.

Two limits apply to every number here. A machine wrote every question and made every grade, and every latency comes from a laptop.

6.6 No novelty detector on the vectors

Decision: no nearest-neighbour novelty score, and embeddings serve search only. The new-template signal of Section 6.4 already answers the question a novelty score asks, the moment a form is mined, with no vector and no threshold to tune. A distance in the vector space would repeat that signal with a threshold added.

Novelty alone is also a weak signal. *O2* software changes across the farm all the time, and a new release prints new forms that are not faults. Detectors that treat every unseen form as an anomaly score 0.23 to 0.27 on F1, where 1 is perfect, once the logs drift from the text they learnt on. Plain principal component analysis scores about 0.71 on the same logs. That is why the counts per template matter more than novelty: a known message that stops or floods is rarely a release. The closest published work on ALICE logs, topic modelling over *InfoLogger* messages [14], is the comparison for any later text detector.

7 Anomaly detection on the metrics

Two store plugins do the work. A *detector*, from the Anomaly Detection plugin, grades how surprising an interval is. A *monitor*, from the Alerting plugin, runs a scheduled search and fires when a condition is true. A detector writes a grade, and a monitor decides whether anyone is paged. Figure 6 in Section 3.6 shows the whole path, from the numbers to one alert.

7.1 What feeds it

Three things matter on a worker: volume, errors and health. Two indices hold them.

The log indices give volume and errors per EPN and family. They also give two lags that we compute ourselves. Every record carries three clocks: when the line was written, when the collector read it, and when the store received it. The ingest pipeline subtracts them:

- **Entry lag**, written to read: how late the EPN’s lines reach the collector.
- **Shipping lag**, read to stored: how long the collector takes to deliver.

Both are read as p95, never the mean. A backlog shows in the slowest records before it moves the average.

The metrics index has two writers, and nothing reaches into a worker to scrape it:

- **Each collector pushes** its own sample every 30 seconds: alive, records out, errors, retries.

- **The poller** samples the cluster and *Dashboards* every 30 seconds, plus a roster check: which collectors were expected to speak and did not.

A dead collector writes nothing. So “collector down” is the poller noticing a missing sample after 90 seconds, not a flag the collector sets.

7.2 Detectors: Random Cut Forest

A detector does not read log lines. Every interval it runs a few aggregations, split per entity, and each result becomes a point in a small feature space: volume, error count, p95 lag, heap. A Random Cut Forest [8] then slices that point cloud at random until every point is isolated. A point in a crowd needs many cuts, and an isolated point needs one or two. Few cuts mean a high grade. Many trees vote, so the grade is stable. The properties that matter here:

- **Streaming.** Normal drifts with the data. A run-start spike alarms briefly, then becomes the new baseline. That is also why a slow leak goes quiet: the model absorbs it.
- **Multivariate.** It flags combinations that never happened, such as throughput collapsing while retries climb, or heap rising while indexing falls.
- **One model per entity.** One noisy EPN does not teach the fleet that noise is normal.
- **Shingles.** Each point joins the last eight intervals, so the model reads a short sequence, not one bucket.
- **Window delay.** It waits one minute on metrics and two on logs, so late records still land in their interval.
- **Warm-up.** About 32 live intervals before a detector leaves initialising, and a few hundred before its grades mean much.

Same failure, two horizons. Every log detector runs twice: at one minute, where eight intervals give eight minutes of context, and at 30 minutes, where they give four hours. The slow twin has the same features. It catches a change the fast model has already learnt as normal. Table 14 lists all seventeen.

Table 14: Seventeen detectors: three on health samples, fourteen on the logs as seven fast and slow pairs.

Detector	Per	Watches
<i>Health samples</i>		
ingest-flow	collector	records out, errors, retries; catches a collector going quiet or retrying before a hard rule trips
node-health	store node	heap, processor, indexing rate, disk; catches heap climbing while indexing falls
dashboards-health	whole cluster	event-loop delay, response time, request rate
<i>Logs, per EPN, each with a 30-minute twin</i>		
il-per-epn	EPN	<i>InfoLogger</i> volume and error count: one EPN silent or erroring while the farm looks fine
central-per-epn	EPN	the same on the central family
local-volume	EPN	volume of the info lines that stay on the worker
il-per-epn-entry-lag	EPN	p95 entry lag on <i>InfoLogger</i>
local-per-epn-entry-lag	EPN	p95 entry lag on local info lines
<i>Logs, per collector, each with a 30-minute twin</i>		
il-collector-shipping-lag	collector	p95 shipping lag on <i>InfoLogger</i>
local-collector-shipping-lag	collector	p95 shipping lag on local info lines

Shipping lag is a collector problem, not an EPN problem, so those two are keyed on the collector. One forecaster, from the same family, predicts each store node’s disk fill a day ahead from a week of hourly points. Disk is the one metric with a real cliff and a smooth approach.

A detector never pages. It writes a grade from 0 to 1 and a confidence into the results index. The monitor ad-high-grade turns that into an alert when both pass 0.7. A high grade means the interval is unusual. It does not mean something is broken.

7.3 Monitors: the cliffs we know

A monitor is a scheduled job inside the cluster. Every minute, or every ten, the store runs the monitor's search, hands the result to a short script, and fires if the script says yes. Nothing is trained: the script is a threshold, a ratio or a presence test. There are two kinds.

Query-level: one question, one answer. cluster-red asks every minute whether any health sample in the last two minutes says red. The cluster is red or it is not, so one fleet alert is the right shape.

Bucket-level: group, then ask per group. collector-down groups the roster samples by collector and fires once per missing one. If two collectors die, two alerts fire, each naming its machine. A query-level monitor could only say "some collector is down". Throttling follows the same split, so one dead collector does not mute a later page about another.

Table 15 lists the hard rules on the health samples.

Table 15: The hard rules catch failures with a known edge, and each names the machine it is about.

Monitor	Fires when	Means
collector-down	an expected collector sent no sample	that machine, or its collector, is gone
collector-unhealthy	the collector is alive but its health check fails	the box is up, the pipeline inside it is sick
fleet-fb-silence	half the roster is silent at once	one shared failure, so one fleet page instead of a storm
shipping-breaking	a collector retried a write and still failed	the pipe is failing; records may still be buffered
data-loss	a collector discarded records	those lines are gone
cluster-red	cluster health is red	a primary shard has no home
shards-stuck	a shard has had no home for 5 minutes	allocation is blocked
disk-cliff-warn, -page	a store node's disk is 85 to 92 %, or above 92 %	approaching the read-only line near 95 %
heap-spiral	heap above 90 % for 5 minutes	not a brief spike: the node is thrashing
admission-rejections	a node refused a search or a write	overload; a refused write drops logs
telemetry-silence	no cluster or <i>Dashboards</i> sample for 5 minutes	the poller is dead

7.4 Trend rules: the baseline a forest never keeps

Static rules catch cliffs and the forest catches unfamiliar shapes, but neither catches a slow drift, because the forest learns it as normal. The trend lane compares against a fixed week instead.

It rests on our own trend rollup service:

1. Every ten minutes it writes one small row per family and entity, a host or, for the lags, a collector: records, errors and fatals, and the p95 and mean of both lags.
2. It waits 120 seconds for late records and rewrites the last three slices. Row identifiers are fixed, so a rewrite never duplicates.
3. A host that logged in the last day and is silent now gets a zero row. The store's own rollup writes nothing for silence, which is why this one is ours.
4. A commit row closes each slice, so no rule reads half a slice.

Nine rules read those rows. Each compares three consecutive ten-minute slices with the previous seven days, leaving out the last 40 minutes so the slices are not part of their own baseline. It fires only when all three slices agree (Table 16).

7.5 Feature engineering was the hard part

The models were the easy choice. Every number fed to them could mislead in its own way:

- **Which machine?** A record names the EPN that wrote it and the collector that shipped it. Keyed on the collector, one model covers a rack and hides a bad EPN. Per-EPN detectors key on the writer.

Table 16: Each trend rule compares a ratio, not a raw count, so a fleet-wide change cancels and one fact raises one alert.

Rule	Compares	Fires when, in all three slices
Volume	the host's share of the fleet	twice the week's share, or half of it; a run start moves every host and cancels
Errors	errors as a share of the host's own records	twice the week's share; doubling traffic and errors together stays quiet
Entry lag	the host's own p95	twice the week's; a slice above one hour is archive age and is ignored
Shipping lag	the collector's own p95	twice the week's; no ceiling, because a long backlog is real

- **Silence is not zero.** An EPN that stops logging produces no point, and the forest ignores it. Volume detectors fill the gap with zero, so silence becomes the loudest thing they see. Lag detectors do not: a missing lag is unknown, not 0 ms.
- **The mean hides a backlog.** So lag is p95. A p95 over a handful of records is just the maximum, so lag rules need 100 records a slice. The plugin reads only the first percentile it is given, so each lag detector asks for exactly one.
- **A run start moves everything.** Shares of the fleet and of the host's own volume cancel it.
- **Replay distorts entry lag.** The replay keeps the original timestamps, so entry lag is the archive's age, about a month. Shipping lag stays real on replay.
- **Small numbers.** Rules need 50 records a slice and 10 errors for an error rule. The floors come from the archive, where daily volume ranges 220 times.
- **Retired hosts.** A collapse needs a host that was busy in the last day, so a decommissioned machine clears itself after 24 hours.

7.6 The signal projector: one history for every alert

The projector turns every alert and every grade into stored, deduplicated signals and episodes, and each episode has a start and an end.

Thirty monitors and seventeen detectors write to different indices, and a firing monitor writes a new row every minute. *Alertmanager* [9] groups and routes well, but it keeps nothing: it forgets an alert five minutes after the last post. Every 30 seconds the projector reads all of it and writes:

- **Signals:** one index for every alert and grade, firing and cleared, named from one catalogue.
- **Episodes:** all signals of one alert name on one entity become one incident with a start and an end. It goes open, recovering, resolved. It closes after one healthy window for a hard rule, three for a trend rule or detector. A missing evaluation is never a recovery, so a lane that stops reporting leaves its episode open and marked stale. When half the roster goes silent at once, one fleet incident opens instead of hundreds.
- **Re-posts:** every open episode goes to *Alertmanager* on every cycle, well inside its five-minute memory.

Table 17: The projector remembers and deduplicates, and *Alertmanager* decides when a person is told.

	Signal projector	<i>Alertmanager</i>
Memory	30 days, replicated	forgets an alert after 5 minutes
Duplicates	thirty rows about one fault are one episode	repeats whatever it is sent
Start and end of a trouble	on every episode	not kept
Read by	the cockpit and the shifter view	the notification receiver
Notification	none of its own	grouping, silences, routing

Why it needed a watermark. Its sources arrive late and out of order: a grade lands after its window closes, a history row when its alert ends, and the projector itself restarts. So it keeps one *watermark* per source, the last moment it fully processed:

1. Read from the watermark minus a short overlap, 15 minutes for alert history and 3 for grades, so late rows are caught.

2. Move the watermark only after a complete read. A failed cycle leaves it, and the next cycle reads the same window again.
3. Build every signal's identifier from its source row, so reading the overlap twice overwrites and never duplicates.

Watchdogs. Every monitor runs inside the cluster, so a dead cluster silences all of them at once, and silence reads as health. Table 18 gives the checks on the detection machinery itself.

Table 18: The watchdogs cover the machinery the other rules depend on, and the two break-glass ones skip the path they report dead.

Check	Fires when	Why it exists
opensearch-unreachable	the projector fails two cluster probes in a row	raised by the projector straight to <i>Alertmanager</i> ; the one alert that survives a dead cluster
signal-projector-stale	the projector wrote nothing for 10 minutes	break-glass: posts straight to the receiver, since the projector is the thing that is dead
alertmanager-down	the projector saw <i>Alertmanager</i> unreachable for 5 minutes	break-glass: nobody is receiving pages
trend-rollup-stale	no complete rollup commit for 40 minutes	every trend rule is reading stale rows
trend-entity-cap	a rollup group nears its host cap	hosts past the cap are silently unwatched

A dead projector and a dead cluster at the same moment, a site or network loss, needs an off-site heartbeat. That does not exist yet.

8 Limits and what remains

8.1 Not measured

Every cost figure in Section 5 comes from a laptop. One micro-benchmark on one farm machine gives the conversion factor. None has run.

The rig gave the worker's collector and store four exclusive cores, and the storage tier four more. Sharing them with reconstruction is not measured.

No run measured a burst. The soak plan's per-worker burst of 10,000 to 20,000 records a second matches the farm-wide peak of 9,781, not a worker's busiest second of 78.

The whole stack's cost was measured at 5,000 records a second only. The collector alone was also costed at 1,000 and 20,000. Round 1's 53,000 was read against a test output that always accepts, so the collector's ceiling with a real store is unknown above 50,000.

The dead-collector chain of Section 3.7 is bounded at about 90 to 240 seconds by cadence, and no run timed it.

A detector needs a few hundred one-minute windows, three to five hours, before its grades mean anything. The injection harness waits 32 one-minute windows. Neither the real warm-up nor that wait was measured on a live feed.

The semantic path was timed on a laptop with a warm index. The collector's 64-chunk burst ceiling is not measured on a worker.

8.2 Limits found in the design

The stale-projector monitor fires only while one projector heartbeat exists in the last 24 hours, so a day of silence mutes it. The *Alertmanager*-down monitor reads the projector's own report, so a dead projector silences it too.

No inhibition rule is active, because all 22 causal edges between alerts ship unproven.

Nothing authenticates inside the cluster, so one password and a firewall rule are the whole boundary, on five machines we own.

The live lane carries *InfoLogger*, the daemon log and the central family only. Process output lines with no severity went central before the fix of Section 5.4, so the live page now shows fewer.

DDS is the largest family during data-taking and the dearest per line. The recipe runs under-sample it at 43,972 lines, 0.1 % of the weight, and the later runs used 1,236,971 lines.

At its 20,000-template ceiling the stamper evicts a template, a rebuilt one restarts its count at one, and the change to keep counting is open.

8.3 What remains, in build order

Seven things remain. Figure 9 shows the three layouts. Staging is two workers and three storage machines. The farm pilot is three workers and three storage containers on one infra machine, and it ran the cluster, the collectors and *Dashboards*. Whether the stamper, the projector and the shifter view ran on the farm is not recorded. Production, the whole farm with at least three storage machines and two replicas, is not real.

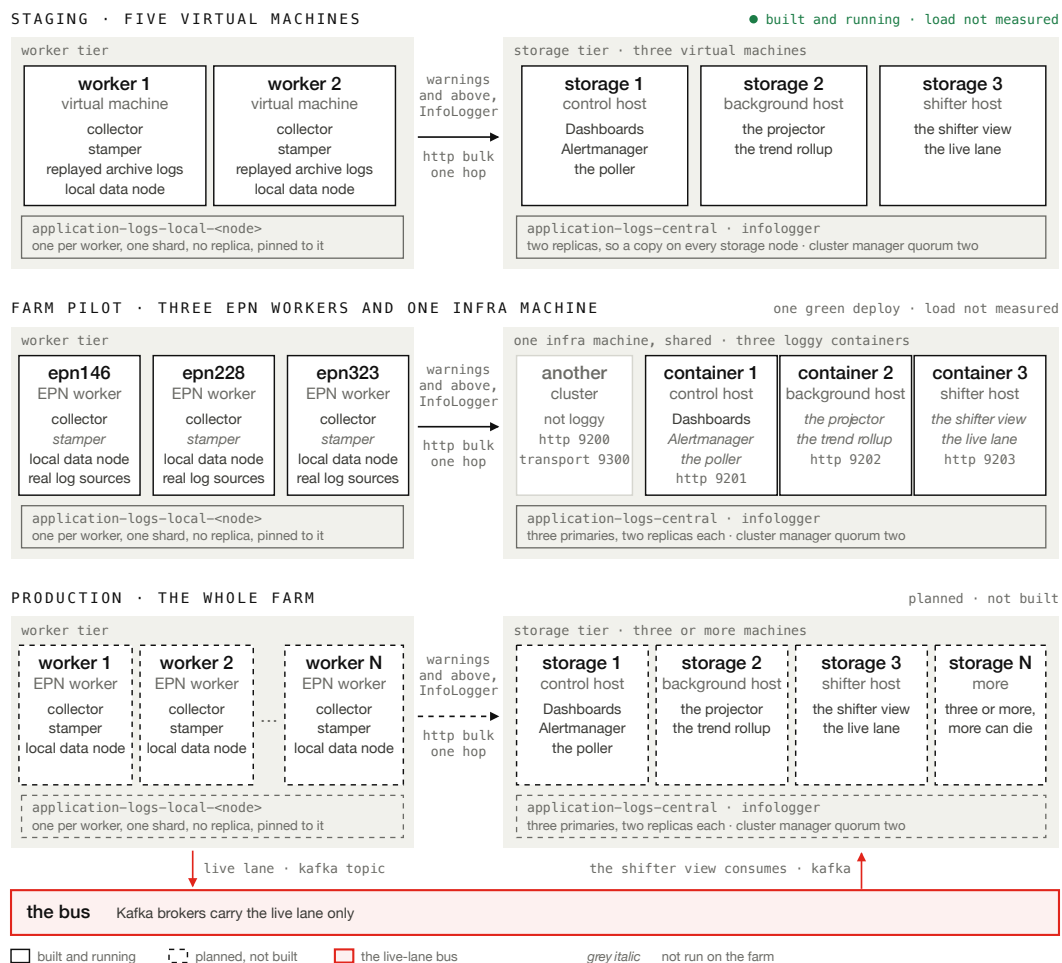


Figure 9: Staging and the farm pilot exist, and production is the same design on the whole farm with a bigger storage tier.

1. The *Telegraf* decision is open: whether CERN's metrics estate takes over the samples.
2. Catalogue maintenance as ISM policies plus monitors is open. ISM, Index State Management, deletes whole indices by age, so the two counting checks need a home.
3. The operator view beside *InfoLogger* waits for farm access and a live run.
4. The system journal and per-device process logs are absent from the archive.
5. Detectors on per-template counts. The bucket counts exist, and no detector or trend rule reads them yet. Nearest-neighbour novelty on the text is closed (Section 6).
6. Authentication inside the cluster is required before the farm.
7. Source-owner approval for reading the node logs is open.

9 Conclusion

The logs an EPN writes can stay where they are made and still be searched as one, on four cores. The watching costs a worker nothing it did not push itself.

A median worker carries 23 records a second in the busiest hour of six months of archive. The whole stack on one worker sustains about 42,000 records a second on the laptop rig. The templating pipeline is 31 to 57 % cheaper per family and byte-identical. Dense retrieval with reranking reaches 0.685 nDCG@10 against 0.634 without it.

One limit binds every figure: all come from a laptop, so shapes and rankings transfer to the farm and absolute rates do not.

Today *loggy* runs on five staging machines, and one farm deployment passed every check. Production, the whole farm with a larger storage tier, is not built yet.

Acknowledgements

I thank my supervisors, Lubos Krcal and Federico Ronchetti, for their direction and reviews. The 2025 reference design by Athanasios Papadopoulos [1] gave the project its starting point.

A Upstream roles weighed and not used

Every deployment role except *Kafka* is our own. The *Kafka* role comes from the deployment of the 2025 reference design. Table 19 records why each other candidate lost.

Table 19: Each upstream role replaces a few stable tasks and hides something the deployment needs.

Component	What it offered	Why it was not used
Store, <i>Dashboards</i> , nginx	Vendor playbooks, two community roles, nginx role	Built around the security plugin we disable. Community roles stale, single-tier. The nginx role owns the site.
Collector	Six community roles	One Debian-only, one without multiline parsers, four only install. The pipeline is the role.
<i>Alertmanager</i> , detection	<i>Prometheus</i> collection role, vendor playbook, <i>Terraform</i> provider	The closest moves the route tree into <i>Jinja</i> . None talks to the detection API. <i>Terraform</i> keeps its own state.
Cockpit metrics, trend rollup	Exporter, <i>Metricbeat</i> , performance analyzer, <i>systemd</i> role, native rollup	Each writes to a store the monitors do not read. A native rollup writes nothing for silence.
Host preparation, template catalogue	Swap, timesync, firewall, kernel roles, ISM, rollover	ISM deletes whole indices by age. Rollover duplicates a catalogue upserted by version.
Replay, faults, ops page, projector, shifter view	None	The software exists only here.

References

- [1] A. Papadopoulos. *Scalable Logging Architecture: expanding InfoLogger into a unified, scalable logging platform*. ALICE Summer Student presentation, CERN, 15 September 2025.
- [2] ALICE O² InfoLogger. <https://github.com/AliceO2Group/InfoLogger>.
- [3] ALICE Collaboration. *Technical Design Report for the Upgrade of the Online–Offline Computing System*. CERN-LHCC-2015-006, ALICE-TDR-019, 2015.
- [4] Fluent Bit documentation. <https://docs.fluentbit.io/>.
- [5] OpenSearch documentation: anomaly detection, alerting, index state management. <https://docs.opensearch.org/>.
- [6] P. He, J. Zhu, Z. Zheng, M. R. Lyu. Drain: an online log parsing approach with fixed depth tree. *IEEE International Conference on Web Services (ICWS)*, 2017.

- [7] Drain3: a streaming implementation of Drain. <https://github.com/logpai/Drain3>.
- [8] S. Guha, N. Mishra, G. Roy, O. Schrijvers. Robust random cut forest based anomaly detection on streams. *International Conference on Machine Learning (ICML)*, 2016.
- [9] Prometheus Alertmanager documentation. <https://prometheus.io/docs/alerting/latest/alertmanager/>.
- [10] Ansible documentation. <https://docs.ansible.com/>.
- [11] Apache Kafka documentation, KRaft mode. <https://kafka.apache.org/documentation/>.
- [12] K. Järvelin, J. Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.
- [13] Cloud Native Computing Foundation, graduated projects and graduation criteria. <https://www.cncf.io/projects/>.
- [14] A. Techaviseschai, S. Tarnpradab, V. Chibante Barroso, P. Phunchongharn. A real-time semi-supervised log anomaly detection framework for ALICE O² facilities. *Applied Sciences*, 15(11):5901, 2025. doi:10.3390/app15115901.
- [15] Fluent Bit documentation, Fluentd and Fluent Bit. <https://docs.fluentbit.io/manual/about/fluentd-and-fluent-bit>.
- [16] Cloud Native Computing Foundation. CNCF announces Fluentd graduation, 11 April 2019. <https://www.cncf.io/announcements/2019/04/11/cncf-announces-fluentd-graduation/>.
- [17] Datadog. Datadog acquires Timber Technologies, the company behind Vector, February 2021. <https://www.datadoghq.com/blog/datadog-acquires-timber-technologies-vector/>.
- [18] Linux Foundation. Linux Foundation announces OpenSearch Software Foundation, 16 September 2024. <https://www.linuxfoundation.org/press/linux-foundation-announces-opensearch-software-foundation-to-foster-open-collaboration-in-search-and-analytics>.
- [19] Elastic. Elastic announces open source license for Elasticsearch and Kibana source code, 29 August 2024. <https://www.businesswire.com/news/home/20240829537786/en/>.
- [20] Elastic. Subscriptions: Elastic Stack features by tier. <https://www.elastic.co/subscriptions>.
- [21] Grafana Loki documentation, label best practices and cardinality. <https://grafana.com/docs/loki/latest/get-started/labels/bp-labels/>.
- [22] OpenSearch documentation, cross-cluster search. <https://docs.opensearch.org/latest/search-plugins/cross-cluster-search/>.
- [23] OpenSearch forum. Is there any limit on number of remote clusters that can be set up for cross cluster search? <https://forum.opensearch.org/t/is-there-any-limit-on-number-of-remote-clusters-that-can-be-setup-for-cross-cluster-search/17385>.
- [24] OpenSearch documentation, anomaly detection security: remote indexes. <https://docs.opensearch.org/latest/observing-your-data/ad/security/>.
- [25] Amazon Web Services. Amazon OpenSearch Service now scales to 1000 data nodes on a single cluster, 19 November 2024. <https://aws.amazon.com/about-aws/whats-new/2024/11/amazon-opensearch-service-scales-1000-nodes-cluster>.
- [26] OpenSearch documentation, exploring data with Discover. <https://docs.opensearch.org/latest/dashboards/discover/index-discover/>.
- [27] OpenSearch documentation, managing OpenSearch Dashboards plugins. <https://docs.opensearch.org/latest/install-and-configure/install-dashboards/plugins/>.
- [28] Prometheus documentation, Alertmanager alerts API: client behaviour. https://prometheus.io/docs/alerting/latest/alerts_api/.
- [29] Redpanda documentation, licenses and enterprise features. <https://docs.redpanda.com/current/get-started/licensing/overview/>.
- [30] Puppet documentation, managing certificate signing requests. https://help.puppet.com/pe/2025.8/topics/managing_certificate_signing_requests.htm.
- [31] HashiCorp. Terraform vs. Chef, Puppet, etc. <https://developer.hashicorp.com/terraform/intro/vs/chef-puppet>.